

Simulación de Entrevista Técnica — Spring Security

Guía de estudio en formato entrevista · uso de **Spring Security + JWT** en el

sistema POS (`auth-service` :8084 · PostgreSQL · jjwt 0.12.6 · BCrypt).

Cada sección incluye la **pregunta del entrevistador**, una **respuesta modelo**

anclada al código real de este proyecto y, cuando aplica, **preguntas de seguimiento** (follow-ups).

Formato sugerido: 45–60 min. Bloques: fundamentos (10'), la SecurityFilterChain

(10'), JWT con jjwt (15'), autenticación y usuarios (10'), operación/seguridad dura (10').

0. Warm-up — Spring Security en el proyecto

P ¿Dónde y para qué se usa Spring Security en este sistema POS?

R Vive en el `auth-service` (:8084), el servicio de autenticación con **PostgreSQL**. Su `build.gradle` incluye `spring-boot-starter-security` más la librería **jjwt 0.12.6** (`jjwt-api` / `jjwt-impl` / `jjwt-jackson`). Cubre tres cosas: **cifrado de contraseñas** con `BCryptPasswordEncoder` , **emisión/validación de JWT** (`JwtService`) y la **configuración del filtro de seguridad** (`SecurityConfig`). Expone `POST /api/v1/auth/login` , `/register` y `GET /api/v1/auth/validate` a través del `AuthController` .

FOLLOW-UP ¿El API Gateway valida el token? — **No** en el código actual: el `api-gateway` solo aplica CORS (`CorsConfig`) y rate limiting; no hay un filtro que verifique el JWT antes de enrutar. Es una brecha conocida (ver sección 5).

1. Fundamentos

P ¿Qué diferencia hay entre autenticación y autorización, y dónde aparece cada una aquí?

R **Autenticación** = quién eres; ocurre en `AuthService.login()`, que valida credenciales contra PostgreSQL. **Autorización** = qué puedes hacer; se modela con **roles y permisos** (`ROLE_ADMIN`, `ROLE_USER`) que se embeben como claims `roles` y `permissions` dentro del JWT. Hoy el proyecto **autentica** de forma sólida pero **no aplica autorización por endpoint** (la cadena permite todo, ver sección 2).

P ¿Por qué `BCryptPasswordEncoder` y no un hash como SHA-256?

R BCrypt es un **hash adaptativo con salt** diseñado para contraseñas: es deliberadamente lento (factor de coste configurable) para resistir fuerza bruta y rainbow tables. SHA-256 es rápido, justo lo contrario de lo que quieres para contraseñas. En el repo se declara como bean en `SecurityConfig.passwordEncoder()` y se usa en `register` (`passwordEncoder.encode(...)`) y en `login` (`passwordEncoder.matches(raw, hash)`).

FOLLOW-UP ¿Se compara la contraseña con `equals`? — No: se usa `passwordEncoder.matches()`, que re-aplica BCrypt con el salt embebido en el hash y compara en tiempo constante.

2. La SecurityFilterChain

P Explica la configuración de `SecurityConfig`.

R Es una clase `@Configuration @EnableWebSecurity` que expone un bean `SecurityFilterChain (@Order(1))`. En él: desactiva **CSRF** (`csrf.disable()`), **httpBasic**, **formLogin** y **logout** porque es una API **stateless**; fija `SessionCreationPolicy.STATELESS` (sin `HttpSession`, sin `JSESSIONID`); configura **CORS** con un `CorsConfigurationSource`; y termina con `authorizeHttpRequests(auth -> auth.anyRequest().permitAll())`.

P ¿Por qué CSRF deshabilitado es aceptable aquí?

R CSRF protege flujos basados en **cookies de sesión** que el navegador envía automáticamente. Esta API es **stateless con JWT en el header Authorization**, que no se envía solo: el atacante no puede forzar al navegador a adjuntarlo. Por eso deshabilitar CSRF es correcto para una API de tokens.

FOLLOW-UP ¿Qué implica **STATELESS**? — Spring Security no crea ni consulta `HttpSession`; cada request se autentica por sí misma (idealmente por el token), lo que encaja con el escalado horizontal de microservicios.

P ¿Ves algún problema en `anyRequest().permitAll()`?

R Sí: **abre todos los endpoints**. Como no hay un filtro JWT que popule el `SecurityContext`, la cadena no autoriza nada por rol/permiso. Para el `auth-service` tiene cierto sentido (login/register deben ser públicos), pero los endpoints de gestión de usuarios (`UserController`) quedan sin protección a nivel de framework. Lo correcto sería `authorizeHttpRequests` con reglas (`permitAll` solo para `/api/v1/auth/**` y `authenticated()` / `hasRole(...)` para el resto) más un filtro que valide el token.

3. JWT con jjwt

P ¿Cómo se genera un token en `JwtService`?

R Con el builder de jjwt 0.12.x: `Jwts.builder()` fija `subject(username)`, añade claims `userId`, `email`, `roles` y `permissions` (roles y permisos se aplanan a CSV desde `user.getRoles()`), `issuedAt`, `expiration (now + jwt.expiration)`, por defecto **86400000 ms** = 24 h) y firma con `signWith(getSigningKey())`. La clave se deriva de `jwt.secret` vía `Keys.hmacShaKeyFor(secret.getBytes(UTF_8))` → algoritmo **HS256**.

P ¿Cómo se valida y se lee el token?

R `validateToken` hace `Jwts.parser().verifyWith(getSigningKey()).build().parseSignedClaims(token)` dentro de un try/catch: si la firma o la expiración fallan, lanza excepción y devuelve `false`. `getUsernameFromToken` parsea y devuelve `claims.getSubject()`. Es firma **simétrica** (HMAC), así que el mismo secreto firma y verifica.

FOLLOW-UP ¿HS256 vs RS256? — HS256 es simétrico (un secreto compartido); simple pero todo el que verifica puede también firmar. RS256 es asimétrico (clave privada firma, pública verifica), preferible cuando varios servicios deben **verificar** sin poder **emitir**. Aquí, si el gateway validara tokens, RS256 evitaría repartir el secreto de firma.

P ¿Cómo protege el proyecto el secreto de firma?

R Con `JwtSecretValidator`, un `ApplicationRunner` que **falla al arrancar** (`IllegalStateException`) si `jwt.secret` está vacío, es el placeholder documentado (`<change_me_at_least_256_bits_long>`) o mide **menos de 32 bytes** (256 bits, el mínimo para HS256). Es un "fail-fast": evita arrancar con un secreto forjable. El valor real llega por la variable de entorno `JWT_SECRET` (`application.yml : jwt.secret: ${JWT_SECRET}`).

FOLLOW-UP ¿Por qué 32 bytes? — HS256 usa HMAC-SHA-256; una clave más corta que el tamaño de bloque debilita la seguridad, y jjwt directamente rechaza claves por debajo de 256 bits.

4. Autenticación, usuarios y roles

P Recorre el flujo de `login`.

R En `AuthApplicationService.login(LoginRequest)`: busca el usuario por `username` (`userRepository.findByUsername`), si no existe lanza `"Invalid credentials"`; valida la contraseña con `passwordEncoder.matches`; comprueba `user.isEnabled()` (si no, `"User account is disabled"`); genera el JWT con `jwtService.generateToken(user)` y devuelve un `AuthResponse` con token, username, fullName, roles y permisos.

FOLLOW-UP ¿Cómo se traducen esos errores a HTTP? — El `@ExceptionHandler` del `AuthController` mapea `"Invalid credentials"` / `"disabled"` → **401**, `"already exists"` → **409** y `"not found"` → **404**.

P ¿El modelo `User` implementa `UserDetails` ?

R **No.** `User` es una `@Entity` JPA pura (tabla `users`) con `username`, `email`, `password`, `enabled`, `createdAt` y un `@ManyToMany` a `Role` (tabla de unión `user_roles`, fetch EAGER). El proyecto **no** usa el flujo estándar de `UserDetailsService` / `AuthenticationManager` de Spring Security: valida credenciales manualmente en el service. Es funcional, pero se aparta del camino idiomático de Spring Security.

FOLLOW-UP ¿Qué ganaría con `UserDetailsService` ? — Integrar `AuthenticationManager`, `@PreAuthorize`, `method security` y un `SecurityContext` poblado, reutilizando la maquinaria del framework.

P ¿De dónde salen el usuario admin y los roles?

R De `DataInitializer` (`ApplicationRunner`): crea `ROLE_ADMIN` y `ROLE_USER` si no existen y, si no hay un usuario `admin`, crea **admin / admin123** (password codificado con BCrypt, rol `ROLE_ADMIN`). Son las credenciales de prueba del stack.

5. Seguridad dura y operación

P ¿Cuál es el mayor riesgo de seguridad del diseño actual?

R Que **el JWT no se valida en el borde**. El `api-gateway` enruta hacia stock/venta/despacho aplicando solo CORS y rate limiting; ningún filtro verifica el token. Un cliente podría llamar a servicios downstream sin un JWT válido si alcanza el gateway o el servicio directamente. La emisión de tokens es sólida (HS256 + secreto validado), pero **falta la aplicación** (enforcement).

FOLLOW-UP ¿Cómo lo cerrarías? — Un `GlobalFilter` / `WebFilter` en el gateway (WebFlux) que valide firma y expiración y propague identidad en headers, o un filtro JWT por servicio; idealmente pasar a **RS256** para que el gateway verifique con la clave pública sin poder emitir.

P El token vive 24 h y no hay logout real. ¿Qué problema hay y cómo se mitiga?

R Un JWT es **stateless**: una vez emitido es válido hasta expirar; no se puede "revocar" sin estado extra. Con 24 h, un token filtrado sirve un día. Mitigaciones: acortar la expiración y añadir **refresh tokens**, o una **denylist** de tokens revocados (encaja con el **Redis** que ya usa el gateway) consultada en cada validación. Hoy `logout` está deshabilitado en `SecurityConfig`, coherente con no tener sesión, pero deja la revocación sin resolver.

FOLLOW-UP ¿Qué claims ayudarían a revocar? — Un `jti` (id único de token) para marcarlo en la denylist, y versionado de credenciales para invalidar todos los tokens de un usuario al cambiar contraseña.

P CORS está en `allowedOriginPatterns("*")` con credenciales. ¿Es seguro?

R Es **permisivo**: en `auth-service` se permite cualquier origen con `allowCredentials(true)`. Para desarrollo local vale, pero en producción se debe restringir a los orígenes reales de los frontends (`:4200` , `:4300` , `:4400`), ya que orígenes comodín + credenciales amplían la superficie de ataque.

Apéndice — Chuleta rápida

Tema	Punto clave (real en el repo)
Ubicación	<code>auth-service</code> :8084, PostgreSQL, <code>spring-boot-starter-security</code> + <code>jjwt</code> 0.12.6
Config	<code>SecurityConfig</code> : <code>@EnableWebSecurity</code> , STATELESS, CSRF/formLogin/basic off, <code>anyRequest().permitAll()</code>
Passwords	<code>BCryptPasswordEncoder</code> (<code>encode</code> en register, <code>matches</code> en login)
JWT emisión	<code>JwtService.generateToken</code> : HS256, claims <code>userId/email/roles/permissions</code> , exp 24h
JWT validación	<code>parseSignedClaims</code> con <code>verifyWith</code> ; <code>subject</code> = username
Secreto	<code>JwtSecretValidator</code> falla si vacío/placeholder/<32 bytes; <code>JWT_SECRET</code> env
Endpoints	<code>AuthController</code> → <code>/api/v1/auth/login\ register\ validate</code>
Errores	<code>@ExceptionHandler</code> : 401 credenciales/disabled, 409 exists, 404 not found
Usuarios	<code>User</code> (JPA, no <code>UserDetails</code>)+ <code>Role</code> / <code>Permission</code> ; <code>DataInitializer</code> : admin/admin123
Brecha clave	El gateway no valida el JWT (solo CORS + rate limit) → falta enforcement
Mejoras	Filtro JWT en gateway, RS256, expiración corta + refresh, denylist en Redis, CORS restringido